Contents

Report Generation Prompt Chain

Use Case

Use this chain when a human analyst needs Edge Copilot to generate a maintainable Python script that extracts Oracle data, transforms it with pandas, and writes a formatted Excel workbook.

Required Local Preparation

- Gather business reporting requirements, workbook layout, sheet names, and expected formatting.
- Prepare SQL or compressed schema context plus extraction intent.
- Know the local Python version, dependency constraints, credential method, and output file path.
- Prepare sample row counts, validation totals, or expected workbook checks.

Prompt Sequence

1. `01-requirements-intake.md`: collect report and runtime requirements.
2. `02-sql-schema-submission.md`: submit SQL, schema notes, and data rules.
3. `03-python-etl-generation.md`: request extraction and pandas transform code.
4. `04-excel-formatting-generation.md`: request workbook writing and styling.
5. `05-script-review-verification.md`: review the full script and local checks.

Related Procedure Closet

- `procedures/report_generator/SKILL.md`

Related Reference Drawers

- `references/python-idioms/sections/pandas_etl.md`

- `references/python-idioms/sections/openpyxl_excel.md`

Report Generation Chain - 01 Requirements Intake

Follow the MDCS procedure closet at `procedures/report_generator/SKILL.md`.

Act as a Senior Python Developer building an Oracle-to-pandas-to-Excel reporting script. This is the first step of a multi-turn report generation workflow.

Ask targeted questions needed before code generation. Cover:

1. Report objective and target audience.
2. Oracle extraction input and bind parameters.
3. Workbook sheets, columns, order, names, and formatting expectations.
4. Runtime environment, dependency constraints, and credential source.
5. Validation checks for row counts, totals, dates, and workbook structure.

Do not write code yet. Point to `references/python-idioms/` only when detailed Python, pandas, or openpyxl guidance becomes relevant.

Report Generation Chain - 02 SQL And Schema Submission

Continue following `procedures/report_generator/SKILL.md`.

I am providing SQL or compressed schema context plus workbook requirements. Review the inputs and identify any gaps before implementation.

SQL or extraction intent:

```
<paste SQL query, query fragments, or extraction intent here>
```

Schema or data notes:

```
<paste compressed schema context, column meanings, bind values, row-volume notes, and dtype co
```

Workbook requirements:

```
<paste sheet names, column order, headers, formatting, grouping, sorting, and output path here
```

Return:

1. Confirmed script requirements.
2. Data extraction and transform plan.
3. Workbook structure plan.
4. Missing details that block reliable code generation.

Report Generation Chain - 03 Python ETL Generation

Continue following `procedures/report_generator/SKILL.md`.

Generate the Python extraction and pandas transformation portions of the report script. Use `references/python-idioms/sections/pandas_etl.md` as the relevant drawer, but do not duplicate tutorial content from it.

Return code that includes:

1. Configuration placeholders for connection settings, SQL, bind parameters, and output paths.
2. Oracle extraction using `python-oracledb` Thin mode with safe cleanup.
3. pandas transformations with explicit dtype and null handling.
4. Clear function boundaries for extraction, transform, and orchestration.
5. Concise assumptions for any unresolved requirement gaps.

Do not add Excel styling yet unless it is required to keep the script coherent.

Report Generation Chain - 04 Excel Formatting Generation

Continue following `procedures/report_generator/SKILL.md`.

Extend the report script with Excel workbook writing and formatting. Use `references/python-idioms/sections` as the relevant drawer, but do not repeat its reference content.

Workbook formatting requirements:

```
<paste confirmed sheet names, column order, formats, widths, freeze panes, filters, highlights
```

Return:

1. Workbook writing code with stable sheet ordering.
2. A separate openpyxl styling pass.
3. Column formats for dates, numbers, IDs, percentages, and text as applicable.
4. Empty-result and file-write behavior.
5. A short note on how styling remains separate from data logic.

Report Generation Chain - 05 Script Review And Verification

Continue following `procedures/report_generator/SKILL.md`.

Review the full generated script and verification results. Focus on reliability, resource cleanup, data correctness, and workbook usability.

Script or relevant excerpts:

```
<paste generated script or changed sections here>
```

Local verification results:

```
<paste run output, workbook checks, row counts, totals, formatting observations, or errors here
```

Return:

1. High-priority issues to fix before use.
2. Required code changes, if any.
3. Verification checklist status.
4. Remaining assumptions or deployment notes.